

Registro para Minutas 1

Estudiante	Minuta 1			
	Tareas realizadas	Resultados obtenidos	Próximo paso	Fecha / Evaluación
Rayan Bentaleb Zaki	- Creación de la arquitectura base del proyecto mediante Programación Orientada a Objetos (src/modelos.py y src/main.py). - Modelado de las clases Pedido (incluyendo cálculo automático del ratio beneficio/peso) y GestorPedidos para centralizar la lógica. - Implementación de la infraestructura de la ciudad (GrafoUrbano) utilizando listas de adyacencia mediante diccionarios para optimizar la memoria.	- Estructura de datos robusta y modular, preparada para recibir las mejoras de ordenación y algoritmos voraces de los compañeros. - Grafo urbano funcional capaz de registrar nodos, conectar aristas de doble sentido y devolver distancias (gestionando nodos no conectados con float('inf')). - Código principal (main.py) depurado, libre de errores de inicialización de atributos y ejecutándose correctamente por consola.	- Supervisar la integración del algoritmo de selección mediante Programación Dinámica (Tema 4) dentro de la estructura de GestorPedidos. - Adaptar los métodos del Grafo Urbano para dar soporte a las estrategias de poda necesarias para el algoritmo de ruteo por Backtracking (Tema 5).	11/04/2026 / Pendiente
Óscar López Rojo	- Pruebas realizadas de código: Desarrollo de comparador_voraz.py. - Pruebas realizadas de código: Implementación de Quicksort personalizado.	- Solución voraz para la mochila (Módulo 2) basada en ratio (beneficio/peso) finalizada. - Algoritmo Divide y Vencerás (Módulo 3) multicriterio operativo.	- Integrar los algoritmos base con las futuras implementaciones de Programación Dinámica y Backtracking de mis	11/04/2026 / Pendiente

	- Búsqueda de información para documentación técnica.	- Análisis de complejidad teórica ($O(n \log n)$) redactado en el informe.	compañeros para iniciar la comparativa de métricas.	
Jorge Sáez Villate	- Creación de los archivos .json de cada escenario correspondiente - Descripción acerca de los 5 tipos de escenarios - Mejora acerca de recursividad (Tema 1) para crear las instancias sobre la prueba (generador_escenarios.py)	- Una estructura de datos con los distintos archivos de los escenarios - Pruebas para probar los algoritmos con casos de cada escenario en específico - Aplicación de recursividad para instanciar objetos y la sincronización mediante diccionarios	- Integrar la lectura automática de los archivos .json generados directamente en el GestorPedidos del main.py - Realizar una medición de tiempos del Backtracking comparando escenarios - Implementar la función de generación de pedidos en el main para poder crearlos de forma automática en vez de forma manual	11/04/2026/Pendiente

Resumen de la Reunión de Coordinación:

- **Asistentes:** Rayan Bentaleb Zaki, Óscar López Rojo, Jorge Sáez Villate
- **Fecha de la reunión:** sábado 11 de abril por la mañana
- **Temas tratados y acuerdos alcanzados:** * Se realizó la lectura y análisis conjunto del guion de la práctica.

-
- Ante la falta temporal de los contenidos de los Temas 4 y 5, el grupo acordó por unanimidad adelantar la Fase 1 (Modelado) y las mejoras obligatorias correspondientes a los Temas 1, 2 y 3.
 - Se estableció la división de roles: Arquitectura del sistema (Rayan), Diseño de Escenarios/Testing (Jorge) y Algoritmia Base (Óscar).

Investigación y Propuesta de Diseño: Para el desarrollo de la arquitectura base, el grupo ha revisado la documentación de los Temas 1, 2 y 3 de la asignatura. Como resultado de esta investigación:

- Se ha descartado el uso de matrices de adyacencia tradicionales para el GrafoUrbano, optando por un modelo de diccionarios de diccionarios (listas de adyacencia) en Python, lo cual optimiza el consumo de memoria en grafos urbanos dispersos.
- Se ha diseñado el diagrama de clases principal separando la entidad de datos (Pedido) de su lógica de negocio (GestorPedidos) para cumplir con los principios de alta cohesión y bajo acoplamiento.

Aportación específica - Arquitectura: Definición de la estructura modular del proyecto y supervisión de la integración de las clases GrafoUrbano y GestorPedidos para asegurar la escalabilidad del sistema.

Aportación específica - Testing: Diseño de los casos de prueba iniciales y del escenario de red urbana, verificando que el sistema gestione correctamente los errores de nodos aislados y la carga de datos.

Aportación específica - Algoritmia: Implementación de la lógica de cálculo de rutas y ratios de eficiencia en el script de prueba, garantizando que el procesamiento de los pedidos sea óptimo bajo la nueva estructura de diccionarios.

Demostración de Código y Pruebas Realizadas: Se ha desarrollado un script de prueba (main.py) para validar la integridad de las estructuras de datos. A continuación, se adjunta la salida por consola que demuestra la correcta creación de pedidos, el cálculo automático de ratios y la validación de distancias en el grafo (incluyendo el manejo de nodos no conectados):

```
=== INICIANDO SISTEMA UAH-ROUTE ===  
  
--- 1. Creación de Pedidos ---  
Estado inicial: [Pedido(id=P001, p=10, b=50), Pedido(id=P002, p=5, b=60), Pedido(id=P003, p=15, b=45)]  
El ratio (beneficio/peso) del pedido P002 es: 12.0  
  
--- 2. Construcción de la Ciudad ---  
Estado del grafo: GrafoUrbano(nodos=4)  
Distancia de Alcalá a Madrid: 30 km  
Distancia de Alcalá a Getafe: inf km  
Nodos conectados directamente a Alcalá: {'Madrid': 30, 'Torrejón': 10}
```